

KEY LEARNINGS REPORT

Introduction

The one-month internship at Raritone Private Limited was an intensive, hands-on experience in backend development. Working on a real-world virtual fashion platform, I gained practical knowledge in Node.js, Express, MongoDB, authentication, API design, security, testing, cloud technologies, and team collaboration. This report summarizes the most valuable lessons I learned.

1. Backend Development with Node.js and Express

What I Learned:

- How to structure a production-ready Node.js application with separate folders for routes, controllers, models, middleware, and utils.
- The request-response cycle in Express: middleware → route → controller → model → response.
- How to use `express.json()` to parse JSON bodies, `cors()` for cross-origin requests, and `compression()` for gzip.
- Building RESTful APIs following standard conventions (GET for retrieval, POST for creation, PUT/PATCH for updates, DELETE for removal).

Practical Example:

I implemented the wishlist module with `addToWishlist`, `removeFromWishlist`, and `getWishlist` endpoints, each with proper HTTP methods and status codes.

Impact:

I can now build a full REST API from scratch, with clean code organization.

2. Database Design with MongoDB and Mongoose

What I Learned:

- Designing schemas for real-world entities: User, Product, Cart, Order, Wishlist, Wardrobe, Measurement, TryOn, Image, ProductMapping.
- Using Mongoose data types (String, Number, ObjectId, Array, Date).
- Creating indexes for performance: email: 1 (unique), userId: 1, category: 1, createdAt: -1.
- Establishing relationships using ref and populating with .populate().

Practical Example:

I designed the wishlist schema with a products array of ObjectIds referencing the Product model. Then I used .populate('products') to return full product details.

Impact:

I understand how NoSQL databases work and when to use embedded documents vs. references.

3. Authentication and Security

What I Learned:

- **JWT:** How to generate access tokens (expires in 15 min) and refresh tokens (7 days) using jsonwebtoken.
- **Password hashing:** Using bcryptjs with 10 salt rounds.
- **Google OAuth:** Integrating google-auth-library to verify ID tokens and create/login users.
- **Role-Based Access Control (RBAC):** Defining USER, ADMIN, SUPER_ADMIN roles and middleware to restrict access.
- **Rate limiting:** Configuring express-rate-limit to 100 requests per 15 minutes.
- **Helmet:** Setting secure HTTP headers (CSP, X-XSS-Protection, etc.).
- **Input validation:** Using Joi schemas to validate req.body before controllers.

Practical Example:

I added authMiddleware to wishlist routes and verified that unauthenticated requests receive 401.

Impact:

I can now implement secure authentication and authorization in any Node.js application.

4. API Development and Documentation

What I Learned:

- Designing consistent API responses: { success, message, data }.
- Using appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 500).
- Handling errors globally with a centralized errorMiddleware.
- Documenting APIs using Postman and Markdown.

Practical Example:

I documented all wishlist endpoints with request/response examples, making it easy for frontend developers to integrate.

Impact:

I understand the importance of clear API documentation for team efficiency.

5. Module-Based Architecture

What I Learned:

- Separating concerns: routes only define endpoints, controllers hold business logic, models define data structure.
- Using middleware for cross-cutting concerns (auth, validation, error handling).
- Keeping configuration in config/ and utilities in utils/.

Practical Example:

I created wishlistController.js, wishlistRoutes.js, and Wishlist.js model. This made the code easy to test and modify.

Impact:

I can build scalable, maintainable applications using modular architecture.

6. Testing and Debugging

What I Learned:

- Using **Postman** to test endpoints manually: creating collections, setting environment variables, writing test scripts.
- Testing edge cases: duplicate wishlist entries, invalid product IDs, expired tokens, missing fields.

- Using `console.log` and `console.time` for performance debugging (e.g., measuring product fetch time).
- Reading error stack traces from `errorMiddleware` to identify issues.

Practical Example:

I tested rate limiting by sending 101 requests in a loop and confirmed the 101st returned 429.

Impact:

I can systematically test and debug backend APIs, reducing bugs before deployment.

7. Cloud Technologies (Cloudinary & MongoDB Atlas)

What I Learned:

- **Cloudinary:** Uploading images via `Multer`, optimizing with `Sharp`, storing `secure_url` and `public_id`, deleting old images on update/delete.
- **MongoDB Atlas:** Setting up a cloud database, whitelisting IPs, using connection strings with environment variables.

Practical Example:

In the wishlist module (though not directly uploading images), I collaborated with team members who handled Cloudinary integration, learning how media pipelines work.

Impact:

I understand how to integrate cloud storage and database services into a backend.

8. Git and GitHub Collaboration

What I Learned:

- Branching strategy: `main` for production, `dev` for integration, feature branches (feature/wishlist).
- Commit best practices: meaningful messages, atomic commits.
- Pull requests and code reviews.
- Resolving merge conflicts when multiple developers edited the same file.

Practical Example:

I created a `feature/wishlist-auth` branch to add authentication middleware, opened a PR, and merged after review.

Impact:

I can work effectively in a team using Git workflows.

9. Scalability Concepts

What I Learned:

- **Caching:** Redis can store product catalog or user sessions to reduce database load.
- **Queues:** BullMQ for async tasks (e.g., AI try-on processing).
- **Load balancing:** Distributing traffic across multiple Node.js instances with Nginx or AWS ELB.
- **Containerization:** Docker for consistent environments, Kubernetes for orchestration.
- **Database scaling:** Indexing, sharding, and read replicas in MongoDB Atlas.

Practical Example:

I researched these technologies and documented them in the scalability section of our report.

Impact:

I know how to plan for high traffic and future growth.

10. AI Integration Planning

What I Learned:

- Designing database schemas that are AI-ready: ProductMapping with supportedBodyTypes, compatibility, avatarCompatibility .
- Creating mock endpoints for AI services (try-on) that return 202 Accepted with a session ID.
- Using [Socket.IO](#) for real-time updates during long-running AI tasks.

Practical Example:

I helped define the TryOn model and contributed to the design of the avatar module.

Impact:

I understand how to prepare a backend for machine learning integration without disrupting existing features.

11. Teamwork and Communication

What I Learned:

- Regular stand-up meetings to sync progress.
- Using project management tools (GitHub Projects, Trello) to track tasks.
- Pair programming to debug complex issues.
- Giving and receiving constructive code reviews.

Practical Example:

When I added wishlist auth, I communicated with Vagdevi (who handled auth middleware) to ensure compatibility.

Impact:

I can collaborate effectively in a software development team.

12. Time Management and Problem-Solving

What I Learned:

- Breaking down large tasks (e.g., wishlist module) into smaller sub-tasks (schema, controller, routes, testing).
- Prioritizing security and data integrity over new features.
- Knowing when to ask for help vs. debugging independently.

Practical Example:

The duplicate prevention issue took 2 hours to solve because I initially overcomplicated it. After stepping back, I found the simple includes() check.

Impact:

I improved my analytical thinking and persistence.

13. Personal Growth and Confidence

What I Learned:

- I can build a complete backend module from scratch – schema, business logic, API endpoints, testing, and documentation.
- I can read and understand other developers' code, make changes, and integrate it.
- I am comfortable with the entire backend development lifecycle.

Impact:

This internship has given me the confidence to take on backend roles in future projects.

Conclusion

The Raritone internship was a transformative learning experience. I moved from theoretical knowledge of Node.js and MongoDB to practical, hands-on implementation of a production-ready system. I am grateful for the opportunity to work with a talented team, solve real challenges, and contribute to an innovative fashion technology platform. The skills I gained – in backend architecture, security, testing, cloud, and collaboration – will serve as a strong foundation for my career as a software developer.
